Entering React after learning JavaScript

A brief introduction to React, its benefits, and why it would help you to learn JavaScript before learning React.

Why learn React?

Innovation

React can be integrated seamlessly with any other library or framework because React is a view only library, (it is the view part of the **M**odel **V**iew **C**ontroler MVC architectural UI pattern). The freedom to use any other framework and library makes it possible to experiment and so to innovate.

Simple API

React has a relatively simple API and so has no steep learning curve. It only has a handful of methods to understand. The code to make 'components' of the UI (i.e., buttons etc) is straight-forward as there is very little boilerplate code which makes the library comparatively accessible.

JavaScript

JavaScript is arguably the world's most popular programming language and is also incredibly powerful. JavaScript is both a functional and an object-oriented programming language and so harnesses the advantages brought by both paradigms. React is in JavaScript and so inherits its benefits.

ES6

"ES6 refers to version 6 of the ECMA Script programming language. ECMA Script is the standardized name for JavaScript, and version 6 is the next version after version 5, which was released in 2011. It is a major

version after version 5, which was released in 2011. It is a major enhancement to the JavaScript language, and adds many more features

intended to make large-scale software development easier." You can read more for the details of the new features that ES6 introduces.

JSX

JSX stands for **J**avascript **S**yntax **E**xtension. It allows you to use HTML and XML in JavaScript. Essentially, the HTML/XML code is 'transpiled' into JavaScript. In other words, the HTML/XML is converted into JavaScript before the code is compiled. This enables the full power of JavaScript to be employed with the relative ease of writing in HTML.

Unidirectional Data Flow

React implements one-way data flow which makes it easy to reason about your app. Essentially all the parts of the UI are contained within each other which makes tracing data flow super simple.

Community

React is a very popular and growing library. This means that the React community is large and growing too. So there are a plethora of resources out there to help you as you learn. Essentially, you won't get 'stuck' anywhere for extended periods of time as someone will have had the same problem as you before and is likely to have posted about it online.

Why bother learning JavaScript? Why not dive straight into React?

While it is possible to learn React without any knowledge of JavaScript, it won't be easy. Frequent run-ins with language specific issues would become roadblocks. If you know JavaScript well, on the other hand, your React journey will be smooth, and the focus would be on utilizing the functionalities of React itself. In the next lesson, we will have a look at the skeleton code that the create-react-app provides and what JavaScript concepts can help.

Variable Declarations in JavaScript

This lesson teaches how variables are declared in JavaScript and what's the difference between var, let and const keywords.

Earlier before 2015, the only keyword available to declare variables was the `var` keyword. With JavaScript ES6 introduced, the two new keywords that came into existence are `const` and `let`. Let's see how each of these keywords differ from one another and what's the best time to use them.

The different variable declarations with `var`, `let` and `const` can be confusing for newcomers to React even though they are not React specific. Maybe it is because JavaScript ES6 was introduced when React became popular. In general, I try to introduce `let` and `const` very early in my workshops. It simply starts with exchanging `var` with `const` in a React component:

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Also, JavaScript didn't have a dedicated keyword to declare constants only which is why `const` was introduced. The variables assigned using `const` are read-only which means that once they are initialized using `const`, they cannot be reassigned.

Scope of var, let and const

The primary reason to introduce two more keywords in the new JS version was to allow programmers to decide scope options for their defined variables.

The scopes of all these keywords are mentioned below:

- `var`: Function in which the variable is declared
- `let`: Block in which the variable is declared
- `const`: Block in which the variable is declared

> *Block scopes are what you get when you use if statements, for statements or write code inside curly brackets.*

Run the following example and see what's the result.

```
function foo(){
  for (var i=0; i<5 ; i++){
    console.log(i);
  }
  console.log(i);
}

foo();
```

If you replace `var` with `let` and then run the code again you will get the error `ReferenceError: i is not defined`. This shows that `i` is only accessible within the curly brackets.

Rules of Thumb:

- Don't use `var`, because `let` and `const` is more specific
- Default to `const`, because it cannot be re-assigned or re-declared
- Use `let` when you want to re-assign the variable in future
- Always prefer using `let` over `var` and `const` over `let`

When to use what?

While `let` is usually used in a `for` loop for incrementing the iterator, `const` is normally used for keeping JavaScript variables unchanged. Even though it is

possible to change the inner properties of objects and arrays when using

`const`, the variable declaration shows the intent of keeping the variable unchanged.

Operators & Expressions in JS

This lesson lists the commonly used JavaScript's expressions and operators, including assignment, comparison, arithmetic, bitwise, logical and string.

Operators

JavaScript operators can be categorized into two main categories i.e., *Unary* and *Binary* operators. Unary takes only one operand whereas binary takes two.

1. Binary Operators

Binary operators can further be divided into following types:

- *Arithmetic Operators* ($+, -, *, /$)

```
//Arithmetic Operators
console.log("****Arithmetic Operators****\n")
console.log("2 + 3 = " + (2 + 3))
console.log("2 - 3 = " + (2 - 3))
console.log("2 + 3 = " + (2 * 3))
console.log("6 + 3 = " + (6 / 3))
console.log("7 + 3 = " + (7 / 3))
```

- *Assignment Operators* (=, +=, -=, *=)

```
//Assignment Operators
console.log("\n****Assignment Operators****\n")
var x = 3;
console.log("x = " + x)
console.log("x += 1 gives x = " + (x+=1)) // adds 1
console.log("x -= 1 gives x = " + (x-=1)) // subtracts 1
console.log("x *= 3 gives x = " + (x*=3)) // multiplies 3 with x
```

- *Logical Operators* ($&&, | |, ! $)

```
//Logical Operators
console.log("\n****Logical Operators****\n")
console.log("1 OR 1 = " + (1 || 1)) // 1 OR 1
console.log("1 OR 0 = " + (1 || 0)) // 1 OR 0
console.log("0 OR 0 = " + (0 || 0)) // 0 OR 0
console.log("1 AND 1 = " + (1 && 1)) // 1 AND 1
console.log("1 AND 0 = " + (1 && 0)) // 1 AND 0
console.log("0 AND 0 = " + (0 && 0)) // 0 AND 0
console.log(!true)  // NOT TRUE
console.log(!1)     // NOT TRUE
console.log(!false) // NOT FALSE
console.log(!0)     // NOT FALSE
```

- *Comma Operator* (,): The Comma operator evaluates each operand from left to right and returns the value of right most operand.

```
//Comma operator
console.log("\n****Comma Operator****")
var a = 4;
a = (a++, a);
console.log("The value for expression with comma operator is: " + a) //returns 5
```

- *Comparison Operators* ($<, >, ==, ! =$)

```
//Comparison operators
console.log("\n****Comparison Operators****")
console.log(1 > 2) //false
console.log(1 < 2) //true
console.log(1 == 1) //true
console.log(1 != 1) //false
```

- *Bitwise Operators* (&, |, ^)

```
//Bitwise Operator
```

```
console.log("\n****Bitwise Operators****")
console.log("Bitwise AND of 5 and 1: " + (5 & 1)) //returns 1


console.log("Bitwise OR of 5 and 1: " + (5 | 1)) // returns 5
console.log("Bitwise XOR of 5 and 1: " + (5 ^ 1)) //returns 4
```

- *String Operators* (+)

```
//String Operator
console.log("\n****String Operator****")
console.log("Concatenation" + " (+)" + " operator in action")
```

- *Conditional Operators* (? :)

```
//Conditional Operator
console.log("\n ****Conditional Operator****")
var num_of_months = 13
var ans = (num_of_months > 12) ? "Invalid" : "Valid"
console.log(ans) //Returns Invalid
```

> **Did you know?**
>
> A comma operator (,) is used when you want to evaluate an expression from left to right.

2. Unary Operators

As mentioned earlier, unary operators take only one operand in order to perform a specific operation. Some of the commonly used unary operators in JavaScript are:

- `typeof` : Returns the type of the given operand

- `delete`: Deletes an object, object's attribute or an instance in an array
- `void`: Specifies that an expression does not return anything
- *Increment Operators* : ++, --

> **Did you know?**
>
> The operator `===` is commonly referred as Deep Equals in JavaScript. The only difference between double equals `==` and deep equals is that the former does not perform type comparison but in fact, converts the type of one of the operands to make their types same. Deep equals, on the other hand, returns false if both types are not the same.

See the example given below for better understanding:

```
console.log(1 == 1); //returns true
console.log('1' == 1); //returns true
console.log(1 === 1); //returns true
console.log('1' === 1); //returns false
```

Expressions

Anything that evaluates to a value is called an *expression*. Some of the basic expressions and keywords used in JavaScript are mentioned below:

- `this`: points to the current object
- `super`: calls methods on an object's parent, for example, call parent's constructor
- `function`: used to define a function
- `function*`: used to define a generator function
- `async function`: used to define an async function

There is one special operator which we have left for now... let's discuss it in the next lesson!

Ternary Operator in React

This lesson teaches the use of Ternary Operator in React for conditional rendering of JSX and explains it with the help of examples.

A ternary operator — also called Conditional Operator — is the only JavaScript operator which takes three operands and returns a value based on some condition. It's an alternative for `if` statement. This could be used for multiple purposes and comes in very handy in React too!

Displaying JavaScript strings, objects, and arrays in React is not enough. What about an if-else statement for enabling *conditional rendering*? You cannot use an if-else statement directly in JSX, but you can return early from the rendering function. Returning `null` is valid in React when displaying nothing. Just like we did in the example given below.

> **Did you know?**
>
> *Conditional rendering* in React uses JavaScript operators like `if` or the conditional operator to create elements representing the current state, and let React show or hide a certain UI element based on a condition.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

However, if you want to use an if-else statement within the returned `JSX`, you can do it by using a JavaScripts ternary operator:

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Another way of doing it, if you only return one side of the conditional rendering anyway, is using the `&&` operator:

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

I will not go into detail why this works, but if you are curious, you can learn about it and other techniques for conditional rendering over here: All the conditional renderings in React. After all, the conditional rendering in React only shows, again, that most of the React is actually JavaScript and not anything React specific.

Object Destructuring & Spread Operators

This lesson teaches the concepts of Object Destructuring and Spread Operators and how we can use them in React.

What is Destructuring?

Another language feature introduced in JavaScript is called *Object Destructuring*. It's often the case that you have to access plenty of properties from your state or props in your component. Rather than assigning them to a variable one by one, you can use *destructuring* assignment in JavaScript. See the following example for better understanding:

Assume there is an object named `student` and you want to extract its properties, one way to do this is:

```
const student = {
  ID: '21',
  name: 'Jhon',
  GPA: '3.0',
};

const id = student.ID;
const name = student.name;
const GPA = student.GPA;

console.log(id);
console.log(name);
console.log(GPA);
```

But as you can see, there's a lot of repetitive code in this example. So, to make things simpler, we can rewrite this code as:

```
const student = {
  ID: '21',
  name: 'Jhon',
  GPA: '3.0',
};
```

```
const {ID, name, GPA} = student;
```

You can list the properties you want inside the curly brackets and even rename them using aliases like this:

```
const student = {
  ID: '21',
  name: 'Jhon',
  GPA: '3.0',
};

const {name:n} = student;
console.log(n);
```

Another example of destructuring could be:

```
// no destructuring
const users = this.state.users;
const counter = this.state.counter;

// destructuring
const { users, counter } = this.state;
```

That's especially beneficial for functional stateless components because they always receive the props object in their function signature. Often you will not use the props but its content, so you can destructure the content already in the function signature.

```
// no destructuring
function Greeting(props) {
  return <h1>{props.greeting}</h1>;
}

// destructuring
function Greeting({ greeting }) {
  return <h1>{greeting}</h1>;
}
```

The destructuring works for JavaScript arrays too. Another great feature is the rest destructuring. It is often used for splitting out a part of an object, but keeping the remaining properties in another object.

```
// rest destructuring
const { users, ...rest } = this.state;
```

Afterward, the users can be used to be rendered, for instance in a React component, whereas the remaining state is used somewhere else. That's where the JavaScript spread operator comes into play to forward the rest object to the next component. In the next section, you will see this operator in action.

Spread Operator

Another JavaScript feature that we use quite a lot in React is the Spread Operator. Spread Operator literally spreads the contents of an array into its elements which makes operations like concatenation etc. easier. Let's say we want to concatenate two arrays, we either do this by using `concat` function, like this:

```
a = [1,2,3];
b = [4,5,6];
c = a.concat(b);
console.log("c: " + c);
```

Or we can do the same thing using Spread Operator:

```
a = [1,2,3];
b = [4,5,6];
c = [...a, ...b]; //spread operator
console.log("c: " + c);
```

Why is Spread Operator so handy in React?

- You can easily add some elements in the middle of the two arrays: `[...a, 'something', ...b];`
- It's simpler to use and also gives you a visual idea of what your array looks like.
- You can clone arrays using this operator: `clone = [...a];`

- In React, you can combine two objects using Spread Operator and add extra properties to that object too. Let's say we have two objects `a` and `b`, we can combine these two objects and create a new one with all the properties present in both objects. See the example below:

```
const person = { name: "Jhon"};
const student = { ID: "21", GPA: "3.0"};

const new_object = { ...person, ...student, semester: '3'};
console.log(new_object);
```

In addition to this, we can also clone objects in React using Spread Operator.

Quick Quiz on JavaScript Variables & Operators!

Test your knowledge of Variables, Expressions and Operators with this short quiz!

1

You can add new properties to an instance object any time later after its creation. True or False?

2

The most recommended way of declaring a variable is using:

3

Which of these are accessible throughout the code:

Which of these are accessible throughout the code.

4

What will be the output of the following code snippet?

```
const student = {
  ID: '21',
  name: 'Jhon',
  GPA: '3.0',
};

const {GPA:n} = student;
console.log(n);
```

5

Automatic type conversion is supported by JavaScript. Is this statement True or False?

Check Answers

Function Declaration in JavaScript

A short introduction to JavaScript functions, the number of ways it can be declared in JavaScript and a few concepts related to it.

A *function* is a piece of code which is defined only once but can be called a countless number of times. A JavaScript function comprises several components which affect its behavior. A typical JavaScript function has the following components:

- the `function` keyword
- the name
- the parameter(s)
- the returned value
- the return type
- the context `this`

> **Did you know?**
>
> *In JavaScript, functions are actually objects. Just like any typical object, they have attributes and methods too. The only thing that differentiates them from objects is that they can be called*

What's the difference between Named and Anonymous function?

The only difference is that *Anonymous Functions* are declared at runtime, means they are defined and called at the same time. The reason why they are called Anonymous is that they are not given a proper name before compilation. A typical way to declare a function in JavaScript is:

```
function myFunction()
```

```
{
  console.log("Hello! I'm a named function!");
}


myFunction();
```

Now here's how an anonymous function is dynamically declared:

```
var myFunction = function()
{
  console.log("Hello! I'm an Anonymous function");
}

myFunction();
```

Ways to Declare JavaScript Function

There are many ways to declare a function in JavaScript. The two most common ways are by using function declaration or by function operator. In the function declaration, the `function` keywords appear before the function name. Whereas if the `function` keyword appears anywhere else, that means we are declaring the function by the function operator method. Given below are the six different methods which are used in JavaScript to declare a function.

- *Function Declaration*
- *Function Expression*
- *Generator Function*
- *Generator Function Expression*
- *Arrow Function*
- *Function Constructor*

These declaration types control how the function interacts with its external components like the outer scope, the context, and the object that owns it. Given below is a table which covers all the types, their syntax and when is suitable to use them:

Name	Explanation	Syntax
Function Declaration	This is the most typical method to declare a function in JavaScript. All functions declared using this method allow hoisting; means they can be used before declaration.	• `function function_name(Arg1, Arg2..){}`
Function Expression	This is the most commonly used type. It is most suitable to use when you want to assign your function as an object to a variable. It's often used when you want to use your function as callback function.	• *Named*: `var var_name = function function_name(Arg1,Arg2..){};` • *Anonymous*: `var var_name = function(Arg1, Arg2..){};`
Generator Function Declaration	It is used to declare a Generator Function, a function that uses `yeild` keyword to return a Generator-Iterator object on which `next` method can be called later.	• `function* name(Arg1, Arg2..) {}`
Generator Function Expression	This is much similar to the type we just discussed above. The only difference is that	• *Named*: `function* function_name(Arg1,Arg2..){}`

	it allows omitting name from the function.	• *Anonymous*: `function* (Arg1,Arg2..){}`
Arrow Function	The two reasons why this type of functions were introduced in ES6 are: writer shorter syntax for function expressions and get rid of `this` value. You can exclude function parentheses if it only takes one parameter. You can also erase the curly brackets if there's only one statement inside function body.	• `var var_name = (Arg1, Arg2..) => {};`
Function Constructor	This is the least recommended way of declaring a function. Here, the `Function` keyword is actually a constructor which creates a new function. The arguments passed to the constructor become arguments of the newly created function and the last parameter is a string which is converted	• `var var_name = new Function(Arg1, Arg2..,'FunctionBodyString');`

	into a function body. This may cause security and engine optimization problems which is why it's always never recommended to use.	

Coding Challenge: Write a JavaScript function expression

Rewrite the `cube()` function given using a 'function expression'. It can be named or anonymous but remember to name the object (i.e., `var` etc) `cube` or your code won't compile.

```
// Add your function expression here
function cube(n) {
}
```

That's pretty much about functions! In the next lesson, we will discuss the Arrow Functions in detail and see what we can achieve through them in React.

Arrow Functions in React

This lesson demonstrates the uses of Arrow Functions in React.

As mentioned in the previous lesson, Arrow Functions are a more concise way of writing a function in JavaScript. They are frequently used in React to make things more efficient and simpler (Event Handling, preventing bugs, etc.).

When teaching someone about React, I explain JavaScript arrow functions pretty early. They are one of JavaScript's language additions in ES6 which pushed JavaScript forward in functional programming.

```
// JavaScript ES5 function
function getGreetingFunc() {
  return 'Welcome to JavaScript';
}

// JavaScript ES6 arrow function with body
const getGreetingArrow1 = () => {
  return 'Welcome to JavaScript';
}

// JavaScript ES6 arrow function without body and implicit return
const getGreetingArrow2 = () =>
  'Welcome to JavaScript';

console.log(getGreetingFunc());
console.log(getGreetingArrow1());
console.log(getGreetingArrow2());
```

Given below is an example to help you understand the importance of arrow functions in JavaScript. Let's say you have an array of student objects with two properties ID's and a boolean to check if that student is present or not. Let's say you want to filter out the students who are absent, one way to do this is:

```
const students = [
  { ID: 1, present: true}
```

```
  { ID: 1, present: true},
  { ID: 2, present: true},
  { ID: 3, present: false},
];

const presentStudents = students.filter(function(student){return student.present;});
console.log(presentStudents);
```

This statement can be easily transformed into a much a simpler and concise syntax by introducing an arrow function, like this:

```
const students = [
  { ID: 1, present: true},
  { ID: 2, present: true},
  { ID: 3, present: false},
];

const presentStudents = students.filter(student => student.present);
console.log(presentStudents);
```

Rules of thumb:

While writing an arrow function or transforming an existing function into arrow function, you must keep these points in mind:

- If there is only one statement inside function body then you can omit `return` keyword as well the curly braces
- Omit the `function` keyword in the beginning
- You can also get rid of parameter parentheses if you only have one parameter

> **Did you know?**
>
> *As arrow function do not rebind `this` so it makes it easier for developers to debug the code and prevent any errors caused by making use of `this` within callbacks. We will cover this in detail in the upcoming chapters.*

JavaScript arrow functions are often used in React applications for keeping the code concise and readable. I love them, teach them early, but always try to refactor my functions from JavaScript ES5 to ES6 functions along the way. At some point, when the differences between JavaScript ES5 functions and JavaScript ES6 functions are clear, I stick to the JavaScript ES6 way of doing it with arrow functions. However, I always see that too many different syntaxes can be overwhelming for React newcomers. So I try to make the different characteristics of JavaScript functions clear before going all-in using them in React.

Coding Challenge: Write a JavaScript arrow function

Rewrite the given square function using arrow functions. Again, remember to name it `square` or your code won't compile.

```
// Write-your-code-here

function square(n){
  return n*n;
}
```

Higher-Order Functions in React

This lesson introduces the key concept of functional programming: High-Order functions, along with their application in React with the help of coding examples.

Higher-order functions are a great programming concept, especially when moving towards functional programming. In React, it makes total sense to know about these kinds of functions, because at some point you have to deal with higher-order components which can be explained best when knowing about higher-order functions in the first place.

Higher-order functions can be showcased in React early on without introducing higher-order components. For instance, let's say a rendered list of users can be filtered based on the value of an input field.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

It's not always desired to extract functions, because it can add unnecessary complexity, but on the other side, it can have beneficial learning effects for JavaScript. In addition, by extracting a function, you make it testable in isolation from the React component. So let's showcase it with the function which is provided to the built-in filter function.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
```

```
  document.getElementById('root')
);
```

The previous implementation doesn't work because the `doFilter()` function needs to know about the query property from the state. So you can pass it to the function by wrapping it with another function which leads to a higher-order function.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Basically, a Higher-Order function is a function which returns a function. By using JavaScript ES6 arrow functions, you can make a higher-order function more concise. Furthermore, this shorthand version makes it more attractive composing functions into functions.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Now the `doFilter()` function can be exported from the file and tested in isolation as pure (higher-order) function. After learning about higher-order functions, all the fundamental knowledge is established to learn more about React's higher-order components.

Extracting these functions into (higher-order) functions outside of a React component can be beneficial for testing React's local state management in isolation as well.

```
import React from 'react';
require('./style.css');
```

```
import ReactDOM from 'react-dom';
import Counter from './Counter.js';


ReactDOM.render(
  <Counter />,
  document.getElementById('root')
);
```

Moving functions around your code base is a great way of learning about the benefits of having functions as first class citizens in JavaScript. It’s powerful when moving your code towards functional programming.

Map, Reduce & Filter in React

In this lesson, we'll demonstrate how the map, reduce, and filter methods are used in JavaScript and applied in React.

What's the best approach for teaching the JSX syntax for React newcomers? Usually, I start out with defining a variable in the `render()` method and using it as JavaScript in HTML in the return block.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

You only have to use the curly braces to get your JavaScript in HTML. Going from rendering a string to a complex object isn't any different.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Usually, the next question then is: How to render a list of items? That's one of the best parts about explaining React in my opinion. There is no React specific API such as a custom attribute on an HTML tag which enables you to render multiple items in React. You can use plain JavaScript for iterating over the list of items and returning HTML for each item.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

I'm sure there were countless times in your life when you've had to iterate over an array in a for-loop, pass the array's elements through a function, and put them in a new array. You'll realize that other than the function, the code to do this is repetitive and can be standardized into a function of its own. Well, today's your lucky day because the `map()` method does exactly that - it creates a new array with the results of calling a user-written function on every element in the calling array. You can use the `map()` method to render elements like in the following code block. Having used the JavaScript arrow function before, you can get rid of the function body and the return statement which leaves your rendered output way more concise.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Pretty soon, every React developer becomes used to the built-in JavaScript `map()` methods for arrays. It just makes so much sense to map over an array and return the rendered output for each item. The same can be applied for custom-tailored cases where `filter()` or `reduce()` make more sense rather than rendering an output for each mapped item.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
```

```
  document.getElementById('root')
);
```

In general, that's how the React developers are getting used to these JavaScript built-in functions without having to use a React specific API for it. It is just JavaScript in HTML.

Coding Challenge: Use the map() method

In the following, write a program using the map function that squares the numbers in the hidden given array called `notSquared`. It contains random integers. Call the new array `squared` or your code won't compile.

```
// Write-your-code-here
squared = -1;
```

Quick Quiz on JavaScript Functions!

Test your knowledge of JavaScript Functions with this short quiz!

1 A function passed into another function as an argument is called:

2 A function that returns an iterator using `yeild` keyword is called:

3

What would the following code do?

```
render(){
   let users = [
  { 'name' : 'Elton Jhon', 'age' : 60 },
  { 'name' : 'Elvis Persley', 'age' : 50 },
  { 'name' : 'Kurt Cobain', 'age' : 24}
];
   return (<div>
     {users.map(user => <p>user.name</p>)}
   </div>)
}
```

4

What would this code do?

```
function coolFunction(val){
    return val? "Cool" : "Not Cool";
}

var val = true;
```

5

What would the following function do?

```
function myFunction(my_array){
    return my_array.filter(function(num, position)){

                                return my_array.indexOf(num)
== position;
                            }
}
```

Check Answers

JavaScript Classes and Object Creation

In this lesson, we briefly look at object-oriented programming concepts in JavaScript like classes and objects.

Background

Classes are relatively new to JavaScript as previously, there was only JavaScript's prototype chain which could be used for inheritance too. JavaScript classes build up on top of the prototypical inheritance giving the whole thing a more straightforward representation. Classes in JavaScript are syntactical sugar over prototype chains. You don't need to understand what a prototype chain or prototypical inheritance is for this course but knowing these concepts would give you some context of classes in the language.

What is a class?

A class is a kind of separate mini-program with a context of its own — methods (functions) and properties (variables). They describe an entity in a way that a computer can understand. Disparate 'instances' (called objects) of a class can be created and treated as variables. Let's examine the following code to learn how classes work in JavaScript.

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  getName() {
    return `${this.firstname} ${this.lastname}`;
  }
}

var me = new Developer('Robin', 'Wieruch');

console.log(me.getName());
```

Class Object Creation & Initializing Class Variables

The entity described in this example is a 'Developer.' A new object can be created with the `new` keyword and the 'constructor' of the class can be called at the same time as on line 12 to assign values to the class's properties, namely the `firstname` and the `lastname`. The `firstname` of the `me` instance of the class `Developer` is assigned the value 'Robin,' and the `lastname` is assigned 'Wieruch.'

Also, various class methods such as `getName()` can be used to read or write the properties of the object. The `console.log()` method prints whatever is passed as an argument to it.

That's all you need to understand React class components. A JavaScript class is used for defining a React component, but as you will see in the next lesson, the React component is only a React component because it inherits all the abilities from the `_React Component class_` which is imported from the React package.

Coding Challenge: Create a JavaScript class

In the following coding challenge, write a class in JavaScript that describes a car with the properties `color`, `model`, `engineCap` (engine capacity), and `registrationNum` registration number and methods `getColor()`, `getModel()`, `setColor()`, and `setModel()`. Remember to use the exact spelling given here or your code won't pass.

```
// Write-your-code-here
```

Object Creation in JavaScript

Classes — as mentioned previously — are relatively new to Javascript. Objects are a lot like classes. Have a look at the code sample below. Here, we define three objects `computer`, `computer2` and `computer 3`. Each of them has the properties `brand`, `RAM` and `clockspeed`. In addition, `computer3` has the method `printRam()`. Note that objects can be defined on one line as `computer` is

defined or can span multiple lines as the other two are.

```
let computer = { brand : 'HP', RAM : '8 GB', clockspeed : "2 GHz"};

// object definitions can have spaces and newlines!
let computer2 = {
  brand : 'HP',
  RAM : '8 GB',
  clockspeed : "2 GHz"
};

// Objects can also have 'functions' called methods
let computer3 = {
  brand : 'HP',
  RAM : '8 GB',
  clockspeed : "2 GHz",

  printRam() {
    console.log(this.RAM)
  }
}
```

You may have noticed the keyword `this` used in the class declarations, we'll study it in depth in the next lesson.

The 'this' Keyword & Binding in JavaScript

In this lesson, we study how the notorious 'this' keyword works in JavaScript in the context of explicit, implicit, new, and global binding.

What is *this*?

Almost all beginner JavaScript programmers struggle with the `this` keyword. The good news is that understanding `this` is actually much easier than it seems. In a nutshell, the value of `this` depends on what context it is used in. So, if it is used in a function, it's value will depend on how that function is invoked, i.e., the call site. Let's go through the ways that `this` can be assigned in JavaScript.

Implicit Binding

When the dot notation is used to call a function that is in an object or an object of a class, we say that `this` was bound implicitly. For example, consider the code sample below:

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  getName() {
    return `${this.firstname} ${this.lastname}`;
  }
}

var me = new Developer('Robin', 'Wieruch');
console.log(me.getName()); // 'this' is me
var hobbit = new Developer('Frodo', 'Baggins');
console.log(hobbit.getName()); // 'this' is 'hobbit'
```

Left of the Dot Rule

Whatever is to the *left of the dot* is what `this` is. For example, we call `getName()` first through the `me` object and hence `firstname` and `lastname` of the `me` object, i.e., 'Robin' Wieruch' get printed. Then we call `getName()` through the `hobbit` object and so `this` refers to that `hobbit` object and prints its attributes.

Explicit Binding

Unlike implicit binding, where the function is part of the object, and so it becomes obvious what `this` is, standalone functions can be *bound* explicitly to objects at call time.

`call()` & `apply()`

Consider the code block below. The function, `printName()` is bound explicitly to the `me` object of the `Developer` class on *line 15* using the `call()` function. Furthermore, if `printName()` is called without any object bound to it, as on *line 18* it prints the first and last names as undefined because `this` *is* undefined. Arguments can be passed to a function using `call()` as on *line 28*. However, if you do not want to pass each argument individually and instead pass all your arguments as an array, you can use the `apply()` function as on *line 31*.

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}

var printName = function() {
  console.log(`My name is ${this.firstname} ${this.lastname}`);
};

var me = new Developer('Robin', 'Wieruch');

// '.call()' can be used to explicitly bind a function to an object
printName.call(me);

// printName() is not bound to an object so 'this' is undefined
printName();
```

```
var printInfo = function(lang1, lang2, lang3) {
  console.log(`My name is ${this.firstname} ${this.lastname} and I know ${lang1}, ${lang2}, a
}


// Create an array of languages
languages = ['Javascript','C++', 'Python'];

// Pass each argument individually by indexing the array
printInfo.call(me, languages[0], languages[1], languages[2]);

// Pass all the arguments in one array to .apply()
printInfo.apply(me, languages);
```

`bind()`

When called on a function, `.bind()` sets a `this` context and returns a new function with a bound `this` context. Consider the code below.

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
}

var printName = function() {
  console.log(`My name is ${this.firstname} ${this.lastname}`);
};

var me = new Developer('Robin', 'Wieruch');

// Here we bind the me object to the printName() function and get a new function called newPr
const newPrintName = printName.bind(me);

// bound newPrintName() prints appropriately
newPrintName();

// unbound printName() prints undefined
printName();
```

New Binding

`this` can be defined explicitly within a function as it can be in a class. Consider the code block below. Try experimenting with the values of `this.firstname` and `this.lastname` in the `printInfo` function and see what happens.

```
var printInfo = function(firstname, lastname, lang1, lang2, lang3) {
  this.firstname = firstname;
  this.lastname = lastname;
  console.log(`My name is ${this.firstname} ${this.lastname} and I know ${lang1}, ${lang2}, a
}

languages = ['Javascript','C++', 'Python'];
printInfo('Robin', 'Wieruch', languages[0], languages[1], languages[2]);
```

Global Context

When `this` is used outside of any context such as a class, function, or object, it refers to the *global object*. The global object in the browser is usually the `window` object. Download the file below, which simply prints the global `this` object, and open it with a browser of your choice. Then examine your browser's console (inspect element > console). It will say something like "window{document:...". That is the global window object that `this` refers to. In the case of a terminal and in our case the global object is undefined.

```
<html>
    <head>
        <script>console.log(this)</script>
    </head>
</html>
```

```
// this is undefined in terminals/command prompts
console.log(this);
```

Caveat: arrow functions and 'this'

Remember Arrow Functions? Let's see how the `this` keyword works with them. Consider the code block below. What do you think should get printed?

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }
```

```
}

// declare getName() outside of the Developer class using arrow functions
var getName = () => console.log(this.firstname);

var me = new Developer('Robin', 'Wieruch');
const printMyName = getName.bind(me);

printMyName();
```

Yep, this gives you an error. This is because methods like `apply()`, and `bind()`, etc. don't have any effect on `this` in an arrow function in Javascript. The value of `this` remains the same as it was when the function was called. If you want to bind to a different value, you need to use a function expression.

Coding Challenge: Fix the error

The following code has a bug. Can you find it and fix it?

```
let me = {
  name: "Robin",
  getName: function(){
    console.log(this.name);
  }
}

var getMyName = me.getName;
getMyName();
```

We'll look at how class inheritance in Javascript works in the next lesson.

Class Inheritance

In this lesson, we discuss the syntax and concept of class inheritance in Javascript.

Inheritance in JavaScript

Usually, classes are used for inheritance in object-oriented programming. Here is a brief refresher on inheritance; inheritance enables new classes to take on the properties and methods of existing classes. A class that another class inherits is called a *superclass* or *base class*. A class that inherits from a superclass is called a *subclass* or *derived class*. In JavaScript, the `extends` keyword can be used to inherit one class from another class as shown in the following example. So, the derived class, `ReactDeveloper`, inherits all the abilities from the base class, `Developer`, and also adds its specialized capabilities, namely the `getJob()` method to it.

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  getName() {
    return this.firstname + ' ' + this.lastname;
  }
}

class ReactDeveloper extends Developer {
  getJob() {
    return 'React Developer';
  }
}

var me = new ReactDeveloper('Robin', 'Wieruch');

console.log(me.getName());
console.log(me.getJob());
```

However, as you will progress in your React journey, you will realize that

inheritance is somewhat limited compared to class composition. We will see how that is in the next lesson.

React favours composition over inheritance

In this lesson, we study why React programmers prefer to use class composition over class inheritance.

Why not inheritance?

Imagine you are making a video game and need two characters: one that can shoot lasers and another that can cast spells. Both characters have a name and health.

You solve this problem neatly with inheritance. As per the following, a class structure with one parent class, the `character`, and two child classes called `caster` and `shooter` can be created.

```
class Character {
  constructor(name){
    this.name = name;
    this.health = 100;
  }
}

class Shooter extends Character{
  constructor(name){
    super(name);
  }

  shoot(){
    console.log(`${this.name}: prepare to die!`);
    this.health--;
  }

   getHealth(){
     console.log(this.health);
   }

}


class Caster extends Character{
  constructor(name){
    super(name);
  }

  cast(){
    console.log(`${this.name}: Avada Kedavra!`);
    this.health--;
  }
```

```
  }

   getHealth(){
     console.log(this.health);
   }

}

Dumbledore = new Caster("Albus Percival Wulfric Brian Dumbledore")
Dumbledore.cast();

DarthVader = new Shooter("Anakin Skywalker");
DarthVader.shoot();
```

Suppose you launched your game and it was a big hit. But now, your game's buyers demand another character that can both shoot lasers and cast spells.

There are two things you can do:

1. Make the `shoot()` and `cast()` functions a part of the parent `Character` class and have the `ShooterFighter` class inherit it. The only problem with this is that the `shooter` and `caster` would inherit `cast()` and `shoot()` which they do not need. This problem is called the Gorilla/Banana problem, i.e., you need a banana but get a gorilla and the entire forest with it because of the class inheritance structure.
2. Make copies of the `shoot()` and `cast()` functions and add them as local methods to the 'ShooterFighter' class but this means that you are copying code - which is bad practice as it defeats the purpose of using classes. This problem is called the duplication by necessity problem.

Composition

So, to get around these issues, we can use composition. *Object composition*, in programming, is just classes containing instances of other classes as attributes to implement the desired functionality instead of inheriting classes. How that is done in React and JavaScript is somewhat advanced, and is beyond the scope of this course.

However, you can check out my course The Road to Learn React to study it. Essentially, inheritance follows the **IS-A** principle - which means that child classes *are* parent classes. So in our example, the `Caster` IS-A `Character`. In

classes are parent classes. So in our example, the `caster` is a `character`. In contrast, composition uses the **HAS-A** principle - so a class 'HAS-A' certain

attribute or functionality. In our example, the 'ShooterFighter' class 'HAS-A' `shoot()` function and 'HAS-A' `cast()` function.

Advantages of composition over inheritance

- Class taxonomy does not have to be defined in advance. This makes the code dynamic and adaptable to change.
- Since lesser changes will have to be made in the code, it will also introduce fewer bugs.
- The code is more reusable

Thus, it is evident that using composition over inheritance has serious advantages. Actually, the developers of Facebook, who also developed React and use it *to* develop Facebook (phew!), claim that there haven't been any use cases where they would recommend creating inheritance hierarchies!

Quick Quiz on JavaScript Classes!

Test your knowledge of JavaScript Classes with this short quiz!

1 In JavaScript, `this` keyword refers to:

2 JavaScript classes are basically objects. Is this statement True or False?

3 ES6 does not support *overriding*. Is this statement True or False?

4

Explicit binding occurs when dot notation is used to invoke a function. Is this statement True or False?

5

When we use `call()` on a fuction, the type of binding that occurs is:

Check Answers

Modularity & Import/Export in React

This lesson explains how modules are created and accessed in JavaScript.

What are Modules?

It is always recommended to define only one class per file to make it strongly cohesive. These files are basically *modules*. So, if there are two classes Person and Student defined in the same file, you can split them into two modules and name them as `Person.js` and `Student.js`. Note that you must place these files inside your `App` directory.

> **Did you know?**
>
> *The classes defined in modules are private by default and can not be accessed by other files that exist inside your project. You can make them public by using import/export statements.*

Fortunately, the JavaScript community settled on one way to import and export functionalities from files with JavaScript ES6 with import and export statements.

However, being new to React and JavaScript ES6, these import and export statements are just another topic which requires explanation when getting started with your first React application. Pretty early you will have your first imports for CSS, SVG or other JavaScript files. The create-react-app project already starts with those import statements:

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
```

```
import "./App.css";
```

It's great for the starter project, because it offers you a well-rounded experience how other files can be imported and (exported). Also the App component gets imported in the src/index.js file. However, when doing your first steps in React, I try to avoid those imports in the beginning. Instead, I try to focus on JSX and React components. Only later the import and export statements are introduced when separating the first React component or JavaScript function in another file.

So how do these import and export statements work? Look at the last line of `myfile.js` to see how the variables `firstname` and `lastname` are exported from `myfile.js` to another. They can then be imported in another file with a relative path to the first file as per the following:

```
const firstname = 'Robin';
const lastname = 'Wieruch';

export { firstname, lastname };
```

So, it's not necessarily about importing/exporting components or functions, it's about sharing everything that is assignable to a variable (leaving out CSS or SVG imports/exports, but speaking only about JS). You can also import all exported variables from another file as one object:

```
const firstname = 'Robin';
const lastname = 'Wieruch';

export { firstname, lastname };
```

Imports can have an alias. It can happen that you import functionalities from multiple files that have the same named export. That's why you can use an alias:

```
const firstname = 'Robin';
const lastname = 'Wieruch';

export { firstname, lastname };
```

Named & Default Exports

This lesson briefly explains the difference between Named and Default exports and how to make to both type of exports in JavaScript.

All the cases discussed in the previous lesson were named imports and exports. But there exists the default statement too. Adding a `default` keyword before any class definition makes it a default thing which we want to export from the given module. It can be used for a few use cases:

- to export and import a single functionality from a module
- to highlight the main functionality of the exported API of a module
- to have a fallback import functionality

Syntax:

- To make a class default export inside a module

```
export default class_name {....}
```

- To import a default export class from a module

```
import class_name from module_name;
```

You can leave out the curly braces for the import to import the default export:

```
const robin = {
  firstname: 'Robin',
  lastname: 'Wieruch',
};

export default robin;
```

Furthermore, the import name can differ from the exported default name. You can also use it in conjunction with the named export and import

statements and import the default or the named exports in another file:

```
const firstname = 'Robin';
const lastname = 'Wieruch';

const person = {
  firstname,
  lastname,
};

export {
  firstname,
  lastname,
};

export default person;
```

You can also spare additional lines and export the variables directly for named exports, like this:

```
export const firstname = 'Robin';
export const lastname = 'Wieruch';
```

You can also import a combination of named and default objects from a given module like this:

```
import React, {Component} from 'react';
```

In React, we use a combination of default and named imports very frequently so you will get to see this line in almost every React file.

These are the main functionalities for ES6 modules. They help you to organize your code, to maintain your code and to design reusable module APIs. You can also export and import functionalities to test them.

Libraries in React

A brief introduction to React APIs and their uses.

React is only the view layer for your application. There is some internal state management offered by React, but apart from this, it is only a component library which renders HTML for your browser. Everything else can be added from APIs (e.g. browser API, DOM API), JavaScript functionalities or external libraries. It's not always simple to choose the right library for complementing your React application, but once you have a good overview of the different options, you can pick the one which fits best to your tech stack.

For instance, fetching data in React can be done with the native fetch API like so. *Lines 2 and 3* define constant variables that return a JSON based on a query from a website (API). *Lines 15-17* in the `componentDidMount()` method (this will be discussed in future lessons) fetch the data and set the title within the state to a title returned in the JSON file from the API given here.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

But it is up to you to use another library to fetch data in React. Axios is one popular choice for React applications:

```
import React, { Component } from 'react';
import axios from 'axios';

class App extends Component {
  constructor(props){
    super(props);
    this.state = {
```

```
      data: null,
    };
  }


  componentDidMount() {
    axios.get('https://api.mydomain.com')
      .then(data => this.setState({ data }));
  }

  render() {
    // JSX
  }
}

export default App;
```

So once you know about your problem which need to be solved, React's extensive and innovative ecosystem should give you plenty of options solving it. There again it's not about React, but knowing about all the different JavaScript libraries which can be used to complement your application.

Quick Quiz On JavaScript Modules!

Test your knowledge of Modules with this short quiz!

1 Which of these scenarios best explain Modularity:

2 The intellisense suggested modules are actually dependencies that exist in:

3

While importing a module in a file, we specify its path from:

4

The modules that are imported without using the path symbol are actually:

5

Which of the following keyword is used to rename the module while importing it

Check Answers

React Class Components

All React websites are made up of components and this lesson gives an overview of how a basic React component is made.

What are Components?

Components are simply the building blocks that make-up what a website looks like. For example, Facebook's like button is a component. Let's try to code up a component.

```
import React from 'react';

export default class App extends React.Component {
  render() {
    return (
      <div> This is a component </div>
    );
  }
}
```

Explanation:

Line 1 of the index.js file imports the React library; this allows you to use all the functionality and features of React. Revisit the modules chapter if you want a refresher on how this works.

Line 3 of the app.js file creates a default export `App` class that inherits the `React.Component` class, again if any of these concepts are unclear, revisit the class inheritance lesson.

Line 4 is the `render()` function. That is what displays the content onto the screen. That's why the render() method is mandatory in React class components: The React Component from the imported React package instructs you to use it for displaying something in the browser. Furthermore, without extending from the React Component, you wouldn't be able to use other lifecycle methods including the `render()` method. For instance, there wouldn't be a `componentDidMount()` lifecycle method, because the component would be

an instance of a vanilla JavaScript class. And not only the lifecycle methods

would go away, but React's state methods such as `this.setState()` for local state management wouldn't be available as well.

Right now, a simple 'This is a component' would get printed. Now to actually *use* your component, write the name of your component in tags as on *line 8* of the index.js file, yep, this is just like HTML! This makes your components reusable anywhere on your website.

However, as you have seen, using a JavaScript class is beneficial for extending the general class with your specialized behavior. Thus you can introduce your own class methods or properties. For example, in the following, we add a `getGreeting()` method as on *line 4* in the app.js file and render it with the `this` notation on *line 11*. Also notice that the `render()` function only returns *one* JSX/HTML element which is usually the `<div> ... </div>` element. Everything has to be nested within it. If you try to return more than one element, you'll get an error.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Using components within components

React allows you to use your custom components within other components. For example, in the following, `App` is used within the component `OtherApp` on *line 8*. By doing this, you can build relatively bug-free complex components with ease. Also, note that **all React components need to start with a capital letter**.

```
import React from 'react';

export default class OtherApp extends React.Component {
  render() {
    return (
      <h1> other app </h1>
```

```
    );
  }
}
```

Caveats

JavaScript classes welcome using inheritance in React, which isn't the desired outcome for React because React favors composition over inheritance. So the only class you should extend from for your React components should be the official React Component.

In the next lesson, we'll see how we can make components more configurable using React Props.

React Props

In this lesson, we learn to modify our components and make them more dynamic using props, which is a fundamental concept in React.

What are Props?

Props are like arguments you can pass to a function. Imagine them to be options that can be passed to a component to customize its functionality. For example, the following will always print 'Welcome to Rect', but if you want it to be able to print whatever greeting you want, you can use props.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

The code block below uses props to pass a greeting. Two greetings are passed as props on *Line 8* of the index.js file where `greeting` and `anotherGreeting` are variables that could be named anything. They are used on *lines 5 and 8* of the app.js file. Essentially, you can access whatever prop you define when using the component you as `this.props.nameGivenToProp` if used like `<componentName nameGivenToProp = 'prop value' secondProp = 'second value' />`.

You can also pass Numbers, Functions & javascript expressions to your prop by using {}. As on *line 24*.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
```

```
  <App greeting = "greeting #1" anotherGreeting = "greeting #2"/>,
  document.getElementById('root')
);
```

Now you know how props work. In the next lesson, we'll discuss state in React.

State in React

What a React component looks like is a reflection of its state - in this lesson, we'll look at how to use it and some potfalls

What is state?

The core of every React component is its state. The state determines what the component looks like, and you can update that as you go. The entire page won't reload if a component's state is updated only the component will - hence, it allows you to create pages that are dynamic and interactive. You can think of a component's state like water's temperature; changing temperature can significantly change what water looks like. If the temperature is brought below 0°C the water will solidify, if it is between 0°C and 100°C it will be a liquid, and if brought above 100°C it will turn into a gas and evaporate. You can change how water behaves and looks just by changing the value of temperature and that is exactly what state is in React, you can change how components behave and look just by changing the state.

Creating a component with state

To be a little less abstract, the state is just an object in a React component that you can save and get data from such as strings and numbers. Let's see how the state is defined in a React component.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Changing state

Changing state

Most beginners try to change state in React by using the assignment operator like so `state. data = newValue` but this is *wrong* and will generate an error! So instead, React provides the `setState()` method that you can use to update state. The function, `setTimeout()`, as in the following, on **line 20** , is used like so `setTimeout(function(){}, timeInMS)`. It 'calls back' the function after the given delay in milliseconds. In this case, it calls back the `setState()` function which changes the components `lastname` to 'hood' after 5 seconds. Also, notice that we had to bund this callback function with `this`. That is because `this` within the function refers to the function itself and not the component, so without binding it, your program would crash.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Caveats of Changing States

There are some pitfalls you should be aware of. `setState()` does not *immediately* update your component. React may decide to update it later in a batch with a bunch of other components. If you want to check if your component has updated, you can use the `componentDidMount()` method which will be touched upon in the following lifecycle methods chapter.

React Lifecycle & Lifecycle Methods

This lesson is a brief introduction to React Lifecycle methods that come in very handy in many situations!

Note that this chapter does not cover every lifecycle method in detail - it merely introduces lifecycle methods and touches upon some of the more important ones. If you want something more in depth, consider buying my course The Road to Learn React.

What is a lifecycle method?

Every React component goes through a lifecycle and lifecycle methods are functions that are provided by React through which we can control what happens when each tiny section of your UI renders, updates, considers re-rendering and then disappears entirely.

`componentWillMount()`

This method is invoked before a component is mounted on the DOM. This happens once in the lifecycle of a component and before the first render.

Common use case:

Before rendering a component, you might want to get ready its props or state. For example, if a user logs in, you may request the server for user information that is to be displayed on the webpage. Hence, before mounting, you send a request to the server for the required information.

However, the React documentation recommends using `constructor()` for any initialisations before the rendering and `componentDidMount()` for the above use case. According to them, componentWillMount() is unsafe and should be avoided in new code.

`componentDidMount()`

This method is invoked when a component is mounted on the DOM.

Common usecase:

As this method is called once after the first render, you may want to initialize the state and props that will be required in the DOM. The server can be requested for information at this point.

`componentDidUpdate()`

This method is invoked when a component is updated.

Common use case: You can use this method to send requests to the server for any update in the state or props of the component that are required for DOM.

`componentWillUnmount()`

This method is called when the component is removed from the DOM.

Common use case:

It is generally used when you want to clear up things related to the component. For example, when a user logs outs of an application, you may want to clear their credentials and the other authorization tokens due to security purposes.

React Component Syntax

In this lesson, we'll study some ways that can make our React component declaration shorter and simpler.

Component syntax evolution

React's way of defining components has evolved. In its early stages, the React.createClass() method was the default way of creating a React class component. Nowadays, it isn't used anymore, because, with the rise of JavaScript ES6, the previously used React class component syntax became the default.

However, JavaScript is continually evolving, and thus JavaScript enthusiasts pick up new ways of doing things all the time. That's why you will often find different syntaxes for React class components. One way of defining a React class component, with state and class methods, is the following:

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import Counter from './app.js';

ReactDOM.render(
  <Counter />,
  document.getElementById('root')
);
```

A shorter way of defining a React class

However, when implementing lots of React class components, the binding of class methods in the constructor and having a constructor in the first place becomes a tedious implementation detail. Fortunately, there is a shorthand syntax for getting rid of both annoyances:

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
```

```
import Counter from './app.js';

ReactDOM.render(

  <Counter />,
  document.getElementById('root')
);
```

By using JavaScript arrow functions, you can auto-bind class methods without having to bind them in the constructor. Also, the constructor can be left out, when not using the props, by defining the state directly as a class property. (Note: Be aware that class properties are not in the JavaScript language yet.) Therefore you can say that this way of defining a React class component is way more concise than the other version.

Now you know why React uses JavaScript classes for defining React class components. They are used when you need access to React's API (lifecycle methods, `this.state` and `this.setState()`). In the following, you will see how React components can be defined differently, without using a JavaScript class, because you may not always need class methods, lifecycle methods, and state.

Functions as Components in React

In this lesson, we study functional stateless components which are a shorter way to create components that don't require to use lifecycle methods and don't need to maintain state

Functional Stateless Components

React uses the best of different programming paradigms. That's only possible because JavaScript is a many-sided programming language. On the object-oriented programming side, React's class components are a great way of leveraging the abilities of JavaScript classes (inheritance for the React component API, class methods and class properties such as this.state). On the other side, there are lots of concepts from functional programming used in React (and its ecosystem) too. For instance, React's functional stateless components are another way of defining components in React. The question which led us functional stateless components in React: What if components could be used like functions?

```
function (props) {
  return view;
}
```

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import {App} from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

It's a function (functional) which receives an input (e.g. props) and returns the displayed HTML elements (view). It doesn't need to manage any state (stateless) and doesn't need to be aware of any methods (class methods, lifecycle methods). Under the hood, the function only needs to use the rendering mechanism of the render() method from React components. That's

rendering mechanism of the render() method from React components. That's when functional stateless components were introduced.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import {Greeting} from './app.js';

ReactDOM.render(
  <Greeting greeting = "hello from a functional stateless component that takes props!"/>,
  document.getElementById('root')
);
```

Functional stateless components are the preferred way of defining components in React. They have less boilerplate, add less complexity, and are simpler to maintain than React class components. However, as of now, both have their right to exist.

Using arrow functions to simplify functional stateless components

Previously, the course mentioned JavaScript arrow functions and how they improve your React code. Let's apply them to your functional stateless components. The previous Greeting component has two different looks with JavaScript ES5 and ES6:

```
import React from 'react';

// JavaScript ES6 arrow function
export const Greeting = (props) => {
        return(<p>{props.greeting}</p>);
}
```

```
import React from 'react';

// JavaScript ES6 arrow function without body and implicit return
export const Greeting = (props) =>
        <p>{props.greeting}</p>;
```

JavaScript arrow functions are a great way of keeping your functional stateless components in React concise. Even more when there is no computation in between and thus the function body and return statement can be left out.

The Create-react-app component

Let's discuss the create-react-app component. By now, most of it will be self-explanatory.

Modules

Remember how default objects can be exported without curly braces? So we import `React` *line 1* (referring to the code given below) as *default* and `component` as *named*. On *line 2* we import the `logo` from the logo.svg file. It was exported as default with the name `logo` from another file already. Finally, on *line 21* we export App as the *default* so it can be imported from any other file without curly braces.

Class component

On *line 5* a class called `App` that inherits `Component` is declared to create our class component. Then the `render()` function is used to return *one* JSX/HTML div. Remember that the `render()` function can only return one JSX/HTML element.

JSX

Lines 8-16 are written in JSX. We can't explain each line in depth, but we can give a general description of what this code block does. It first creates a div called App. Then it creates a header which contains the imported logo and the line "Welcome to react". The black part of the output image shown below is essentially the header. Then it simply outputs a paragraph with the line "To get started, edit src/App.js and save to reload."

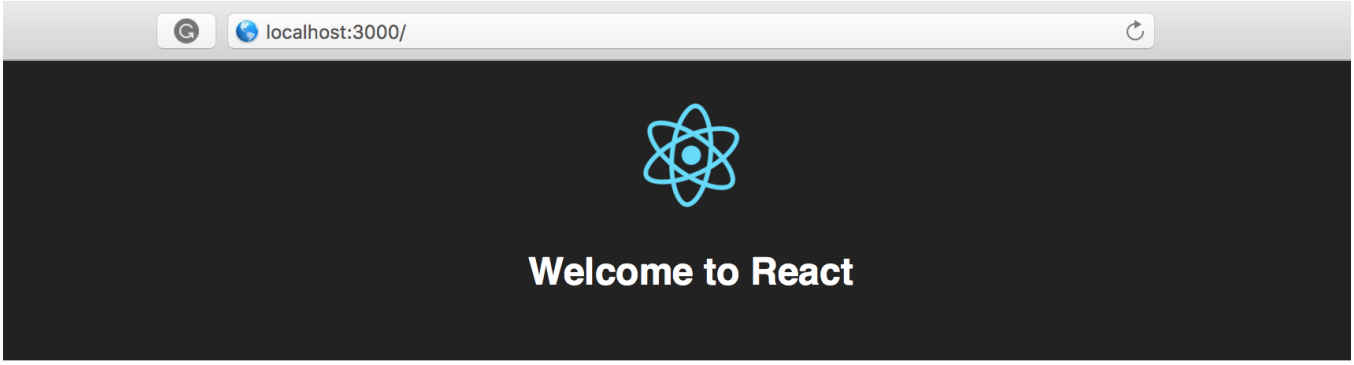
```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
```

```
class App extends Component {
  render() {
    return (

      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
      </div>
    );
  }
}

export default App;
```

Output



To get started, edit `src/App.js` and save to reload.

Output of the react-create-app skeleton code

Quick Quiz on React!

Test your knowledge of React with this short quiz!

1 What is a React component in JavaScript?

2 There is a method in React component which is overidden to prevent updating the components. Which of the following is that method?

3 The data is passed to a component through:

4 How would you write a JavaScript function inside JSX?

5 Calling `setState` method inside `render()` does not generate any errors.

Is this statement True or False?

6 JSX is not a type-safe language. Is this statement True or False?

7 The main purpose of using lifecycle methods is:

8 What happens when you make multiple `setState()` calls within a function?

function?

9 Can components refer to other components? Answer in yes or no.

10 The main focus of React is:

11

The `div` element that is defined inside a render method is an actual DOM element. Is this statement true or false?

Check Answers

Setting up the create-react-app

In this lesson, we Install node and npm to set up a create-react-app project and have a look at a React class component

Installing node

If you are developing with JavaScript, there is no way around node and npm. You can find the installation instructions on the official website. Installing node will install `npm` (node package manager) as well. The node package manager is used to install libraries/frameworks (node packages) using the command line for your project whereas the 'node' command itself can be used to compile and run a JS file in a terminal, i.e.

```
$ node example.js
```

There exists an alternative to install and manage node: nvm. You should be able to install it via Homebrew for MacOS, and there exists a Windows version for nvm too. I highly recommend using nvm, because you can install and manage multiple node versions with it. Furthermore, it helps you to install your node version as well.

Starting a create-react-app Project

Follow the official create-react-app setup guide and read the file `App.js` which contains skeleton code for a React App. It should look something like the following.

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
```

```
class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
      </div>
    );
  }
}

export default App;
```

As you can see this *React class component* might be quite the enigma for someone who does not have much experience with JavaScript even though it is the barebones of a React project. Arguably, a React class component may not be the best starting point. There is loads of non-React-specific information to digest for newcomers: class statements, class methods and inheritance.

Also, the import statements are only adding complexity when learning React. So, the following lessons will shed some light on the fundamental JavaScript building blocks that you would need to understand code like this without worrying about React. We will revisit this component at the end of this course, and by that point, hopefully, it will be easy to understand. In the next lesson, we'll study the types of variable declarations in JavaScript.

There is more JavaScript than React

Final notes by the author to his readers!

In conclusion, there is lots of JavaScript which can be harnessed in React. Whereas React has only a slim API surface area, developers have to get used to all the functionalities JavaScript has to offer. The saying is not without any reason: “being a React developer makes you a better JavaScript developer”. Let’s recap some of the learned aspects of JavaScript in React by refactoring a higher-order component.

```
function withLoading(Component) {
  return class WithLoading extends {
    render() {
      const { isLoading, ...props } = this.props;

      if (isLoading) {
        return <p>Loading</p>;
      }

      return <Component { ...props } />;
    }
  }
  };
}
```

This higher-order component is only used for showing a conditional loading indicator when the isLoading prop is set to true. Otherwise it renders the input component. You can already see the (rest) destructuring and the spread operator in action. The latter can be seen for the rendered Component, because all the remaining properties from the props object are passed to the Component.

The first step for making the higher-order component more concise is refactoring the returned React class component to a functional stateless component:

```
function withLoading(Component) {
  return function ({ isLoading, ...props }) {
```

```
  return function ({ isLoading, ...props }) {
    if (isLoading) {
      return <p>Loading</p>;
    }

    return <Component { ...props } />;
  };
}
```

You can see that the rest destructuring can be used in the function's signature too. Next, using JavaScript ES6 arrow functions makes the higher-order component more concise again:

```
const withLoading = Component => ({ isLoading, ...props }) => {
  if (isLoading) {
    return <p>Loading</p>;
  }

  return <Component { ...props } />;
}
```

And adding the ternary operator shortens the function body into one line of code. Thus the function body can be left out and the return statement can be omitted.

```
const withLoading = Component => ({ isLoading, ...props }) =>
  isLoading
    ? <p>Loading</p>
    : <Component { ...props } />
```

As you can see, the higher-order component uses various JavaScript and not React relevant techniques: arrow functions, higher-order functions, a ternary operator, destructuring and the spread operator. That's how JavaScript's functionalities can be used in React applications.

Conslusion:

Often people say that learning React has a steep learning curve. But it hasn't when only leaving React in the equation and leaving all the JavaScript out of it. React doesn't add any foreign abstraction layer on top as other web frameworks are doing it. Instead you have to use JavaScript. So hone your JavaScript skills and you will become a great React developer.

Want to learn more React?

Lastly, I give a brief overview of my next course "Road to React".

If you are planning to dive deeper and learn more React, my course “Road to Learn React” is a great option! It’s a self-paced course which is designed to help you learn the fundamentals of React and answer all the What’s, Why’s and How’s that you might come across while building a React application. It covers the general concepts, environment setups, patterns and best practices used in a real-world React application. We will also discuss the real-world features like pagination, client-side caching and interactions such as searching, and sorting and learn how to implement them with React!

For more details, you can visit the course contents and details section — Good Luck!